26/04 Inline Functions

When we declare a function as inline, it replaces its definition in the place of its function call

```
inline void Print()
{    cout<<"Hello";
}

int main()
{        int x=10;
         x++;
         Print();
         cout<<x<<endl;

}

inline void Add(int a,int b)
{
         cout<<a+b<<endl;
}
int main()
{        int n1=10,n2=20;
         Add(n1,n2);
}
```

=>

```
int main()
{        int x=10;
         x++;
         {
         cout<<"Hello";
         }
         cout<<x<<endl;

}

int main()
{        int n1=10,n2=20;
         {  cout<<n1+n2<<endl;  }

}
```

We can only request the compiler for inline, its completely the compilers choice whether to consider the request or not.
If request is not accepts it acts like normal function.

We dont have any methodology to check the inline function replaced or not as it takes place at compilation time.

syntax :
inline returndatatype funcnmame()
{

}

Function Overloading:

Function overloading is a compile time polymorphism where we can give same name to more than one function with different signature.

What is the signature of a function?

name of the function
type of arguments
order of arguments
no of arguments

to have a different signature for function overloading functions either no of arguments
order of arguments
type of arguments
should be different.

examples of Function Overloading

```
int Add(int,int);
float Add(float,float);
double Add(double,double);
```

Add2intintint
Add2floatfloatfloat
Add2doubledoubledouble

```
float Add(int,int);
```

Compile time overloading

return datatype of the function does not play any role in the function overloading

In Function overloading compiler will rename the overloaded functions based on no of args, type of args and order of args and this process is called NameMangling.

Default arguments to functions
using default arguments to functions we can give default values to formal arguments, we values not passed then defaul values will be consider.so that instead of multiple definitions we go single definition but we have to fix minimum and maximum no of arguments.

We can make all arguments as default arguments
default value can be anything based on requirement.
default values should be mentioned in the decalartion of the function
if function definition is defined above the main , then supply in the definition.
to make any arguments as default argument , mkae sure that arguments on the right of its all should be default arguments
ex :

```
int fun(int a,int b=0,int c)
{
}
```

